LKY99 审计报告

智能合约安全审计报告
- 免责声明与保密声明
- 1. 项目概览
  - 1.1 审计范围
  - 1.2 审计简介
  - 1.3 项目背景
- 2. 审计总结
  - 2.1 漏洞统计
  - 2.2 审计结论
- 3. 技术与业务分析
  - 3.1 技术快速检查
  - 3.2 业务风险分析
- 4. 代码质量与安全性
  - 4.1 代码质量
  - 4.2 文档与可读性
  - 4.3 依赖使用
  - 4.4 中心化风险
- 5. 审计发现
  - 5.1 严重级别定义
  - 5.2 详细问题
  - Critical
  - High
  - Medium
  - Low / Informational
- 附录 A：完整 PoC 代码（可直接运行）
- 6. 审计方法

智能合约安全审计报告

Prepared by: Audit911**Date:** 2026年4月20日

免责声明与保密声明

> **本报告为安全审计文档，可能包含敏感信息。**
> 文档中包含潜在漏洞与恶意逻辑分析结果，仅建议内部使用；在问题修复前不建议

> 公开披露。
>
> **Audit911** 已基于行业最佳实践对目标合约进行分析，但审计不构成对合约“绝对安全”的保证。投资者/用户仍应自行完成尽调并承担决策风险。

1. 项目概览

1.1 审计范围

Item	Description
Project Name	LKY99
Website	N/A
Platform	BSC
Language	Solidity
Smart Contract Code	`0x28dA7cdEC01Eb82c66cBb37A17376De25e125231`
Commit Hash	N/A（链上合约 + 本地反编译/源码快照）
Audit Date	2026-04-20

1.2 审计简介

本次审计目标是验证 LKY99 合约核心交易逻辑与资金流安全性，重点检查买卖税、AMM 交互、可重入风险、标准兼容性与可观测性问题，并对可利用性进行验证（含 Fork PoC）。

1.3 项目背景

LKY99 是一个带买卖税与分配逻辑的 ERC20 代币，部署于 BSC，并与 PancakeSwap V2 交易对进行流动性交互。合约包含：转账税分发、每日增发、代币买卖封装函数及加池函数。

2. 审计总结

根据本次审计评估，目标合约整体安全性为 **Insecure（存在高风险经济逻辑漏洞）**。

2.1 漏洞统计

Severity	Count
Critical	0
High	1

Severity	Count
Medium	1
Low	2
Informational	0

2.2 审计结论

未发现可直接“无权限盗走全部资产”的传统 Critical 漏洞，但发现 1 个高危经济逻辑问题：买入税从 Pair 直接扣除并分发/销毁，导致用户实际到账与 AMM 报价严重偏离，交易公平性受损并可被用于价格干预。另有重入防护实现缺
陷与若干低风险标准兼容问题。

---

3. 技术与业务分析

3.1 技术快速检查

Main Category	Subcategory	Result
Contract Programming	Solidity version too old	Moderated（使用 `^0.8.0`，建议锁定）
	Integer overflow/underflow	Passed（0.8+ 默认检查）
	Function input parameters lack check	Moderated（多数有检查，部分业务参数依赖外部调用方）
	Access control management	Passed（本报告不将中心化权限纳入漏洞评级）
	Reentrancy / Race condition	Moderated（存在伪重入锁问题）
Code Specification	Function visibility explicitly declared	Passed
	Unused code	Passed
Gas Optimization	"Out of Gas" Issue	Passed
	High consumption loops	Passed

3.2 业务风险分析

Category	Result
Buy/Sell Tax	当前链上观测：买入 3%，卖出 10%
Can Mint?	Yes（每日增发逻辑存在）

Category	Result
Blacklist Function?	No（未发现标准黑名单）
Honeypot Risk?	Potential（高税与到账偏离会形成强交易摩擦）
Anti-Whale/Bot?	Not Detected
Hidden Owner?	Not Detected
Can Take Ownership?	No（仅 owner 可转移 ownership）
Auditor Confidence	Medium

4. 代码质量与安全性

4.1 代码质量

代码结构较直观，逻辑集中在 `_transfer`、`_distributeFees`、买卖与加池函数。
主要问题不在语法层面，而在代币经济实现与 AMM 交互方式。

4.2 文档与可读性

代码注释基础可用，但对“税从何处扣除”“报价与实际到账差异”等关键风险缺少明确说明，易误导用户预期。

4.3 依赖使用

合约主要通过接口调用 Pancake Router/Pair 与 USDT；未使用 `SafeERC20` 包装，存在返回值处理不一致问题。

4.4 中心化风险

按委托要求，本次不将中心化能力纳入漏洞评级。
仅事实披露：owner 具备税率、分配地址、白名单、mint、claimToken 等管理权限。

5. 审计发现

5.1 严重级别定义

Level	Description
Critical	可直接导致资金被盗、资产永久损失或系统失控。
High	对交易安全或资产定价有重大影响，具备现实可利用性。
Medium	应尽快修复，可能在特定条件下造成损失或系统异常。

Level	Description
Low	标准兼容、工程实现或可观测性问题，影响较小。

5.2 详细问题

Critical

未发现 Critical 漏洞。

High

买入税从 Pair 直接扣除并分发/销毁，导致到账偏离与价格干预风险

Description: 买入路径下（`sender == pancakePair`）手续费从 `sender` 扣除，即直接从 Pair 扣减并分发/销毁。Fork PoC 已验证：Pair 按完整 `amountOut` 出币，但买家仅收到税后净额，差额由 Pair 侧流向 burn/费地
址。该机制导致实际成交价格显著劣化，并可被利用进行价格影响放大。

```
  if (sender == pancakePair) {
      feeAmount = (amount * buyFeeRate) / 100; // 买入税
  }

  if (feeAmount > 0) {
      _distributeFees(sender, feeAmount);       // sender=p
air 时，直接从 pair 扣税
      _basicTransfer(sender, recipient, amount - feeAmoun
t); // 用户仅拿净额
  }

  // sources/Contract.sol
  function _distributeFees(address sender, uint256 feeAmoun
t) private lockTheSwap {
      // burn / node / leader / foundation 全部从 sender 扣
      _balances[sender] -= burnAmount;
      _totalSupply -= burnAmount;
      ...
      _balances[sender] -= foundationAmount;
```

```
        _balances[foundationAddress] += foundationAmount;
    }
```

风险说明：

- 买入时 sender = pair，税费直接由 pair 扣出并分发/销毁。
- AMM 结算按 pair 实际出币总量变化更新价格，但用户只收到税后净额，造成实际成交价格显著恶化。
- 这会放大买方不利成交，且可用于价格引导/干预（PoC 已验证）。

PoC 关键实测（BSC Fork）：

- quotedLkyOut: 1,121,316.581999...
- buyerActuallyReceived: 1,087,677.084539...
- lkyTakenFromPairAsFee: 33,639.497459...
- quoteBefore(100 USDT): 347,144.009910...
- quoteAfter(100 USDT): 346,676.156627...（变差）

Recommendation: 将手续费来源改为“买家实际接收额中的扣减逻辑”或采用标准 FOT 兼容模型，确保 AMM 输出与用户到账的一致性；同时新增集成测试校验 `pairΔ = userReceived + feeToX + burn` 的业务一致性。
Status: Open

Medium

`lockTheSwap` 为伪重入锁，外部调用后仍有状态写入路径

Description: `lockTheSwap` 仅设置 `inSwap=true/false`，未进行 `require(!inSwap)` 保护；`buy/sell/addLiquidity` 存在外部调用后继续修改状态的路径。当前固定交互对象下可利用性有限，但模式本身不安全。

```
modifier lockTheSwap {
        inSwap = true;
        _;
        inSwap = false;
    }
```

Recommendation: 改为标准 `nonReentrant` 保护（或等价状态机），并遵循 CEI

（Checks-Effects-Interactions）重排关键逻辑。
Status: Open

Low / Informational

`addLiquidityToPool` 直接改余额未完整发出 ERC20 `Transfer` 事件（Low）

Description: 用户入账与退款使用直接 `_balances` 修改，导致链上事件与真实余额变化不完全一致。

```
_balances[msg.sender] -= tokenAmount;
_balances[address(this)] += tokenAmount;
// 无 Transfer 事件
...
_balances[address(this)] -= unusedToken;
_balances[msg.sender] += unusedToken;
// 无 Transfer 事件
```

Recommendation: 涉及余额迁移时统一走 `_basicTransfer` 或补齐 `Transfer` 事件。
Status: Open

`claimToken` 未检查 ERC20 `transfer` 返回值（Low）

Description: 对非标准 ERC20 可能出现“转账失败但函数不回滚”的静默失败。

```
otherToken.transfer(owner, balance); // 返回值未检查
```

Recommendation: 使用 `require(otherToken.transfer(...), "...")` 或 `SafeERC20.safeTransfer`。
Status: Open

附录 A：完整 PoC 代码（可直接运行）

```
foundry.toml:

  [profile.default]
  src = "poc/src"
  test = "poc/test"
```

```
  out = "poc/out"
  cache_path = "poc/cache"
  solc_version = "0.8.20"
  optimizer = true
  optimizer_runs = 200
  via_ir = true
  evm_version = "paris"

  poc/test/LKY99PairDrainPoC.t.sol（完整）:

  // SPDX-License-Identifier: MIT
  pragma solidity ^0.8.20;

  interface Vm {
      function createSelectFork(string calldata rpcUrl) ext
ernal returns (uint256);
      function deal(address who, uint256 newBalance) extern
al;
  }

  interface IERC20 {
      function balanceOf(address account) external view ret
urns (uint256);
      function approve(address spender, uint256 amount) ext
ernal returns (bool);
  }

  interface IPancakePair {
      function getReserves() external view returns (uint112
reserve0, uint112 reserve1, uint32);
      function token0() external view returns (address);
  }

  interface IPancakeRouter02 {
      function WETH() external view returns (address);

      function getAmountsOut(uint256 amountIn, address[] ca
lldata path)
```

```
            external
            view
            returns (uint256[] memory amounts);

        function swapExactETHForTokensSupportingFeeOnTransfer
Tokens(
            uint256 amountOutMin,
            address[] calldata path,
            address to,
            uint256 deadline
        ) external payable;

        function swapExactTokensForTokensSupportingFeeOnTrans
ferTokens(
            uint256 amountIn,
            uint256 amountOutMin,
            address[] calldata path,
            address to,
            uint256 deadline
        ) external;
    }

    interface ILKY99 is IERC20 {
        function totalSupply() external view returns (uint25
6);
        function pancakePair() external view returns (addres
s);
        function buyFeeRate() external view returns (uint25
6);
        function nodeDividendAddress() external view returns
(address);
        function leaderAddress() external view returns (addre
ss);
        function foundationAddress() external view returns (a
ddress);
    }

    contract LKY99PairDrainPoC {
```

```
      Vm internal constant vm = Vm(address(uint160(uint256
(keccak256("hevm cheat code")))));

      address internal constant LKY99 = 0x28dA7cdEC01Eb82c6
6cBb37A17376De25e125231;
      address internal constant PAIR = 0x7e3D2748a5f0394793
9417f405e8Cfb262788c91;
      address internal constant USDT = 0x55d398326f99059fF7
75485246999027B3197955;
      address internal constant ROUTER = 0x10ED43C718714eb6
3d5aA57B78B54704E256024E;

      event PocResult(
          uint256 buyFeeRate,
          uint256 usdtSpent,
          uint256 quotedLkyOut,
          uint256 buyerActuallyReceived,
          uint256 pairLkyDecrease,
          uint256 lkyTakenFromPairAsFee,
          uint256 feeWalletsLkyGain,
          uint256 totalSupplyBurned,
          uint256 quoteBefore,
          uint256 quoteAfter,
          uint256 pairReserveLkyBefore,
          uint256 pairReserveLkyAfter
      );

      receive() external payable {}

      function test_BuyTaxDrainsPairAndMovesPrice() externa
l {
          vm.createSelectFork("https://bsc-dataseed.binanc
e.org");

          ILKY99 token = ILKY99(LKY99);
          IPancakeRouter02 router = IPancakeRouter02(ROUTE
R);
          IPancakePair pair = IPancakePair(PAIR);
```

```
        IERC20 usdt = IERC20(USDT);

        require(token.pancakePair() == PAIR, "unexpected
pair");

        vm.deal(address(this), 5 ether);
        address[] memory pathBnbToUsdt = new address[]
(2);
        pathBnbToUsdt[0] = router.WETH();
        pathBnbToUsdt[1] = USDT;
        router.swapExactETHForTokensSupportingFeeOnTransf
erTokens{value: 1 ether}(
            0,
            pathBnbToUsdt,
            address(this),
            block.timestamp + 1200
        );

        uint256 usdtBalance = usdt.balanceOf(address(thi
s));
        require(usdtBalance > 100 ether, "not enough USDT
for PoC");

        uint256 usdtIn = usdtBalance / 2;

        (uint256 reserveLkyBefore, ) = _pairReservesLkyUs
dt(pair);
        uint256 pairLkyBalanceBefore = token.balanceOf(PA
IR);
        uint256 totalSupplyBefore = token.totalSupply();

        address node = token.nodeDividendAddress();
        address leader = token.leaderAddress();
        address foundation = token.foundationAddress();
        uint256 feeWalletsBefore = _sumUniqueBalances(tok
en, node, leader, foundation);

        uint256 quoteUsdt = 100 ether;
```

```
          uint256 quoteBefore = _quoteOut(router, USDT, LKY
99, quoteUsdt);

          address[] memory pathUsdtToLky = new address[]
(2);
          pathUsdtToLky[0] = USDT;
          pathUsdtToLky[1] = LKY99;

          uint256 quotedLkyOut = router.getAmountsOut(usdtI
n, pathUsdtToLky)[1];

          require(usdt.approve(ROUTER, type(uint256).max),
"approve failed");

          uint256 buyerLkyBefore = token.balanceOf(address
(this));
          router.swapExactTokensForTokensSupportingFeeOnTra
nsferTokens(
              usdtIn,
              0,
              pathUsdtToLky,
              address(this),
              block.timestamp + 1200
          );
          uint256 buyerLkyAfter = token.balanceOf(address(t
his));

          uint256 buyerActuallyReceived = buyerLkyAfter - b
uyerLkyBefore;
          uint256 pairLkyBalanceAfter = token.balanceOf(PAI
R);
          uint256 pairLkyDecrease = pairLkyBalanceBefore -
pairLkyBalanceAfter;
          uint256 takenFromPairAsFee = pairLkyDecrease - bu
yerActuallyReceived;

          uint256 totalSupplyAfter = token.totalSupply();
          uint256 totalSupplyBurned = totalSupplyBefore - t
```

```
otalSupplyAfter;

          uint256 feeWalletsAfter = _sumUniqueBalances(toke
n, node, leader, foundation);
          uint256 feeWalletsGain = feeWalletsAfter - feeWal
letsBefore;

          (uint256 reserveLkyAfter, ) = _pairReservesLkyUsd
t(pair);
          uint256 quoteAfter = _quoteOut(router, USDT, LKY9
9, quoteUsdt);

          uint256 buyFeeRate = token.buyFeeRate();
          uint256 nominalFee = (quotedLkyOut * buyFeeRate)
/ 100;

          require(buyerActuallyReceived < quotedLkyOut, "bu
yer did not receive less than quoted output");
          require(takenFromPairAsFee > 0, "no LKY taken fro
m pair as buy fee");
          require(_absDiff(takenFromPairAsFee, nominalFee)
<= 2, "fee not taken from pair as expected");
          require(totalSupplyBurned > 0 || feeWalletsGain >
0, "fee not burned/distributed");
          require(quoteAfter < quoteBefore, "price quote di
d not worsen after buy");

          emit PocResult(
              buyFeeRate,
              usdtIn,
              quotedLkyOut,
              buyerActuallyReceived,
              pairLkyDecrease,
              takenFromPairAsFee,
              feeWalletsGain,
              totalSupplyBurned,
              quoteBefore,
              quoteAfter,
```

```
                reserveLkyBefore,
                reserveLkyAfter
            );
        }

        function _quoteOut(IPancakeRouter02 router, address t
okenIn, address tokenOut, uint256 amountIn)
            internal
            view
            returns (uint256)
        {
            address[] memory path = new address[](2);
            path[0] = tokenIn;
            path[1] = tokenOut;
            return router.getAmountsOut(amountIn, path)[1];
        }

        function _pairReservesLkyUsdt(IPancakePair pair) inte
rnal view returns (uint256 reserveLky, uint256 reserveUsdt)
{
            (uint112 r0, uint112 r1, ) = pair.getReserves();
            if (pair.token0() == LKY99) {
                reserveLky = uint256(r0);
                reserveUsdt = uint256(r1);
            } else {
                reserveLky = uint256(r1);
                reserveUsdt = uint256(r0);
            }
        }

        function _sumUniqueBalances(ILKY99 token, address a,
address b, address c) internal view returns (uint256 sum) {
            sum = token.balanceOf(a);
            if (b != a) {
                sum += token.balanceOf(b);
            }
            if (c != a && c != b) {
                sum += token.balanceOf(c);
```

```
        }
    }

    function _absDiff(uint256 x, uint256 y) internal pure
returns (uint256) {
        return x >= y ? x - y : y - x;
    }
}

运行命令：

forge test --match-test test_BuyTaxDrainsPairAndMovesPric
e -vvvv
```

---

6. 审计方法

1. **Manual Code Review：** 对转账、税收分发、买卖/加池、增发与提取逻辑进行逐函数审查。
2. **Vulnerability Analysis：** 基于 AMM 结算模型分析交易公平性、价格影响与可利用路径。
3. **Automated Tools：** 使用 Slither 做静态检测，使用 Foundry Fork 测试做链上行为验证（PoC）。
4. **Reporting：** 对每个发现给出成因、影响、复现依据与修复建议。

© 2026 Audit911. All rights reserved.